🎓 College Database Management System

Database Management System Project Report

Subject: Database Management Systems

Made by: Arav & Apurrv

Section: CS-A

Date: March 2026

1. Introduction

This project implements a **College Database Management System** — a full-stack web application that models the real-world relationships between departments, instructors, courses, students, and enrollments in a college.

> **Problem Statement:** A college contains many departments. Each department can offer any number of courses. Many instructors can work in a department, but an instructor can work only in one department. For each department, there is a head, and an instructor can be head of only one department. Each instructor can take any number of courses, and a course can be taken by only one instructor. A student can enrol for any number of courses and each course can have any number of students.

2. Objectives

- Design a normalized relational database schema that accurately represents college entities and their relationships
- Implement a backend REST API for data retrieval
- Build a modern, visually stunning frontend with interactive animations
- Demonstrate database concepts: primary keys, foreign keys, joins, many-to-many relationships, and referential integrity

3. Technology Stack

LAYER	TECHNOLOGY	PURPOSE
Database	MySQL 8.0	Relational database management
Backend	Node.js + Express.js	REST API server (port 5000)
Frontend	React 19 + Vite 6	Single Page Application (port 5173)
Styling	Vanilla CSS	Dark glassmorphism design system
Animations	ReactBits.dev	WebGL backgrounds, 3D cards, text effects
Icons	Flaticon Uicons (CDN)	Professional vector icon font
Font	Inter (Google Fonts)	Variable font (100-900 weight axis)
Motion	Motion (Framer Motion)	Spring physics animations
WebGL	OGL	Iridescent background shader rendering

4. Database Design

4.1 Entity-Relationship Model

The database consists of 5 tables with the following relationships:

RELATIONSHIP	TYPE	IMPLEMENTATION
Department → Instructor	One-to-Many	FK `dept_id` in instructor table
Department → Head	One-to-One	FK `head_id` (UNIQUE) in department table
Instructor → Course	One-to-Many	FK `instructor_id` in course table
Department → Course	One-to-Many	FK `dept_id` in course table
Student ↔ Course	Many-to-Many	Join table `enrollment` with composite unique key

4.2 Schema Definition (DDL)

Department Table

```
CREATE TABLE department (
    dept_id INT PRIMARY KEY AUTO_INCREMENT,
    dept_name VARCHAR(100) NOT NULL UNIQUE,
    head_id INT UNIQUE DEFAULT NULL
);
```

Instructor Table

```
CREATE TABLE instructor (
    instructor_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
```

```
    dept_id INT NOT NULL,
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)
        ON DELETE CASCADE
);
```

Department Head Constraint

```
ALTER TABLE department
    ADD CONSTRAINT fk_head
    FOREIGN KEY (head_id) REFERENCES instructor(instructor_id)
        ON DELETE SET NULL;
```

Note: This is added after the instructor table exists to resolve the circular dependency between department and instructor.

Course Table

```
CREATE TABLE course (
    course_id INT PRIMARY KEY AUTO_INCREMENT,
    course_name VARCHAR(150) NOT NULL,
    credits INT NOT NULL DEFAULT 3,
    dept_id INT NOT NULL,
    instructor_id INT NOT NULL,
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)
        ON DELETE CASCADE,
    FOREIGN KEY (instructor_id) REFERENCES instructor(instructor_id)
        ON DELETE CASCADE
);
```

Student Table

```
CREATE TABLE student (
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    phone VARCHAR(15),
    semester INT NOT NULL DEFAULT 1
);
```

Enrollment Table (Many-to-Many Join Table)

```sql
CREATE TABLE enrollment (
    enrollment_id INT PRIMARY KEY AUTO_INCREMENT,
    student_id INT NOT NULL,
    course_id INT NOT NULL,
    enrollment_date DATE NOT NULL,
    grade VARCHAR(2) DEFAULT NULL,
    FOREIGN KEY (student_id) REFERENCES student(student_id)
        ON DELETE CASCADE,
    FOREIGN KEY (course_id) REFERENCES course(course_id)
        ON DELETE CASCADE,
    UNIQUE KEY unique_enrollment (student_id, course_id)
);
```

4.3 Sample Data

ENTITY	COUNT	EXAMPLES
Departments	3	Computer Science, Mathematics, Physics
Instructors	6	Dr. Ananya Sharma, Dr. Rajesh Kumar, Dr. Priya Mehta, etc.
Courses	8	Data Structures, Machine Learning, Linear Algebra, Quantum Mechanics, etc.
Students	15	Aarav Verma, Diya Nair, Ishaan Gupta, Meera Joshi, etc.
Enrollments	30	Various student-course pairings with dates and grades

4.4 Key Database Concepts Demonstrated

- **Primary Keys:** Auto-incrementing IDs on all 5 tables
- **Foreign Keys:** 6 foreign key constraints enforcing referential integrity
- **CASCADE Delete:** Deleting a department removes all its instructors, courses, and enrollments
- **SET NULL:** Deleting a head instructor sets department `head_id` to NULL
- **UNIQUE Constraints:** Prevents duplicate emails, department names, and duplicate enrollments
- **Many-to-Many:** Student-Course relationship via enrollment join table with composite unique key
- **Circular Dependency Resolution:** `ALTER TABLE` used to add FK after both tables exist
- **JOIN Queries:** Backend uses multi-table JOINs with GROUP BY for aggregated data

5. Backend API Architecture

The backend is a Node.js Express.js server that provides 6 REST API endpoints. It connects to MySQL using the `mysql2` library with connection pooling.

5.1 API Endpoints

METHOD	ENDPOINT	DESCRIPTION	SQL USED
GET	/api/stats	Dashboard counts	COUNT(*) on all tables
GET	/api/departments	Departments with aggregates	LEFT JOIN + GROUP BY
GET	/api/instructors	Instructors with details	JOIN + subquery
GET	/api/courses	Courses with instructor/counts	JOIN + GROUP BY
GET	/api/students	Students with enrollment count	LEFT JOIN + GROUP BY
GET	/api/enrollments	All enrollments with names	Multiple JOINs

5.2 Example SQL Query (Departments endpoint)

```
SELECT d.dept_id, d.dept_name,
       i.name AS head_name,
       COUNT(DISTINCT i2.instructor_id) AS instructor_count,
       COUNT(DISTINCT c.course_id) AS course_count
FROM department d
LEFT JOIN instructor i ON d.head_id = i.instructor_id
LEFT JOIN instructor i2 ON i2.dept_id = d.dept_id
LEFT JOIN course c ON c.dept_id = d.dept_id
GROUP BY d.dept_id;
```

6. Frontend Design & Implementation

6.1 Design Philosophy

The frontend follows a **dark glassmorphism** design pattern — dark semi-transparent cards with frosted glass effects over a vivid animated background. This creates a premium, modern aesthetic.

6.2 Application Structure

The app is a single-page application (SPA) with 6 views:

1. **Dashboard** — 5 animated stat cards with counters
2. **Departments** — 3 department cards with head, instructor count, and course count
3. **Instructors** — Data table with department badges and head indicators
4. **Courses** — Data table with credits, department, and student count
5. **Students** — Data table with semester badges and enrolled course count
6. **Enrollments** — Data table with grades and enrollment dates

6.3 ReactBits Animated Components

The frontend uses 7 components from reactbits.dev:

COMPONENT	WHAT IT DOES	HOW IT WORKS
Iridescence	Full-screen animated color wave background	WebGL shader (OGL library) renders swirling iridescent colors. Responds to mouse movement. Runs on GPU.
MagicBentoCard	Mouse-tracking purple border glow on cards	Uses CSS custom properties (`--glow-x/y`) updated on mousemove. Radial gradient mask creates the glow.
TiltedCard	3D perspective tilt on hover	Calculates `rotateX/Y` from mouse position. Uses spring physics from Motion library for smooth animation.
PillNav	Pill-shaped nav with hover animation	Circle element expands from bottom on hover. Label text slides up and replacement slides in. Uses `animate()` API.
TextPressure	Title text weight changes near cursor	Each character is a `<span>` . On each animation frame, calculates distance to cursor and interpolates `font-weight` (100→900).
CountUp	Animated number counting	Uses Motion library's `useInView` and `useSpring` to animate from 0 to target number with easing.
GradientText	Animated gradient on footer names	Uses `background-clip: text` with animated `background-position` to sweep gradient across text.

6.4 Additional Frontend Features

- **Flaticon Uicons:** Professional vector icons loaded via CDN (replaced emojis)
- **Inter Variable Font:** Google Fonts loaded with full weight axis (100-900) for TextPressure effect
- **Responsive Design:** CSS grid + media queries for mobile adaptation
- **Page Transitions:** AnimatePresence from Motion for smooth tab switching

- **Staggered Animations:** Cards and table rows animate in sequentially
- **Text Shadows:** Dark text-shadows on header/footer for readability on bright background

7. How to Run the Project

Prerequisites

- Node.js v18+ (nodejs.org)
- MySQL 8.0+ (dev.mysql.com)

Step 1: Setup Database

```
mysql -u root -p < database/schema.sql
mysql -u root -p < database/seed.sql
```

Step 2: Start Backend

```
cd server
npm install
# Create .env file with DB_HOST, DB_USER, DB_PASSWORD, DB_NAME
node index.js
# ✓ Server runs on http://localhost:5000
```

Step 3: Start Frontend

```
cd client
npm install
npm run dev
# ✓ Frontend runs on http://localhost:5173
```

Step 4: Open Browser

Navigate to **http://localhost:5173** to use the application.

8. Project Structure

```
College-db/
├── database/
│   ├── schema.sql          # 5 tables with FK constraints
│   └── seed.sql            # Sample data (3+6+8+15+30 records)
├── server/
│   ├── index.js            # Express REST API (6 endpoints)
│   ├── .env                # MySQL credentials
│   └── package.json        # express, mysql2, cors, dotenv
├── client/
│   ├── index.html          # CDN links (Fonts, Flaticon)
│   ├── vite.config.js      # Dev server proxy config
│   ├── src/
│   │   ├── main.jsx        # React DOM entry
│   │   ├── App.jsx         # Main app (390 lines)
│   │   ├── index.css       # Design system (520 lines)
│   │   └── components/     # 8 animated components
│   └── package.json        # react, motion, ogl
├── .gitignore
├── README.md
└── PROJECT_REPORT.html     # This report
```

9. Key Takeaways

- **Database Normalization:** The schema is in 3NF — no redundancy, all non-key attributes depend on the primary key
- **Referential Integrity:** Foreign keys with CASCADE and SET NULL maintain data consistency
- **Many-to-Many Resolution:** The enrollment join table with composite unique key prevents duplicate enrollments
- **Circular Dependencies:** Resolved using `ALTER TABLE` to add FK after both tables exist
- **REST API Design:** Clean separation of concerns — database ↔ API ↔ frontend
- **Modern Frontend:** React SPA with WebGL, variable fonts, and spring physics animations
- **Full-Stack Integration:** Vite proxy forwards `/api/*` requests to Express backend

10. Conclusion

This project successfully demonstrates the implementation of a relational database system using MySQL with a modern full-stack web application. The database schema accurately models real-world college entities with proper constraints and relationships. The frontend, enhanced with ReactBits animated components, provides an immersive and professional user experience while serving as a practical tool for browsing college data.

Made by **Arav & Apurrv** (CS-A) · March 2026